

AN AGENTIC MULTI-MODAL UNSTRUCTURED DATA PREPROCESSING FRAMEWORK WITH LLM-GUIDED ORCHESTRATION AND QUALITY-DRIVEN VALIDATION

Silva A. S

IT22549136

BSc (Hons) degree in Information Technology Specializing in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

AN AGENTIC MULTI-MODAL UNSTRUCTURED DATA PREPROCESSING FRAMEWORK WITH LLM-GUIDED ORCHESTRATION AND QUALITY-DRIVEN VALIDATION

Silva A. S

IT22549136

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in
Information Technology Specialized in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

DECLARATION

I declare that this dissertation is my own work and does not incorporate without acknowledgement any material previously submitted for a degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

NAME	STUDENT ID	SIGNATURE	DATE
Amodhya Sharinda Silva	IT22549136		22 / 04 / 2026

The above candidate has carried out research for the Bachelor's degree dissertation under my supervision.

Name of the Supervisor: Dr. Lakmini Abeywardhana

Signature of Supervisor: _____

Date: 22 / 04 / 2026

Name of the Co-Supervisor: Dr. Mahima Weerasinghe

Signature of Co-Supervisor: _____

Date: 22 / 04 / 2026

DEDICATION

This dissertation is dedicated to all data engineering practitioners, machine learning researchers, and software architects whose tireless efforts to improve the quality, efficiency, and fairness of data pipelines motivate work such as this. It is also dedicated to the academic community of the Sri Lanka Institute of Information Technology, whose commitment to fostering technical innovation and research excellence has been an enduring source of inspiration throughout this journey.

To every researcher working at the intersection of agentic AI, multi-modal data processing, and streaming systems — this work aims to make a meaningful contribution to the problems you are solving.

ACKNOWLEDGEMENT

The authors acknowledge the data engineering practitioners who provided operational feedback during the design of the framework, particularly on the governance workflow and the taxonomy of preprocessing patterns encountered in production pipelines. Their practical insights were instrumental in shaping the design decisions that distinguish this framework from purely theoretical contributions.

Grateful acknowledgement is extended to the open-source communities behind Apache Kafka, LangChain, Ollama, spaCy, NLTK, OpenCV, scikit-image, and the Pydantic project. Without the tireless contributions of these communities, the implementation described in this dissertation would not have been feasible within a research context. The documentation, examples, and issue trackers maintained by these projects served as indispensable reference materials throughout the development process.

Deep appreciation is extended to the foundational researchers whose prior work made this framework possible. The ReAct framework demonstrated that language models can plan and act in closed loops; HuggingGPT showed that LLM-based orchestration extends naturally to multi-modal settings; Wang et al.'s SSIM metric provided the perceptually grounded quality assessment mechanism that underpins the image validation layer; and Pertuz et al.'s systematic analysis of focus measure operators provided the empirical grounding for the Laplacian variance blur detector used throughout this work.

The MongoDB, FastAPI, and React communities likewise contributed infrastructure that enabled real-time monitoring and persistence within the pipeline. Special thanks are also due to the contributors of the Kafka ecosystem whose work on consumer group semantics and partition management made the horizontal scaling properties of the system architecture possible without modifications to the processing layer.

Finally, sincere thanks are extended to all individuals who provided support, encouragement, and constructive critique during the preparation and evaluation of this research. Your contributions, whether technical, editorial, or motivational, have been deeply valued and have materially contributed to the quality of the final work.

ABSTRACT

Data preprocessing remains a persistent bottleneck in production machine learning pipelines, particularly when inputs are heterogeneous in modality, quality, and linguistic origin. Fixed-sequence preprocessing pipelines apply the same operations regardless of whether a given input actually requires them, wasting computational resources, introducing unnecessary latency, and risking degradation of data that was already of acceptable quality. This dissertation presents an agentic multi-modal preprocessing framework that addresses these fundamental inefficiencies through two complementary and deliberately asymmetric decision mechanisms.

On the text side, a locally-hosted large language model (LLM) deployed via the Ollama inference server dynamically inspects each input record and produces a structured JSON plan specifying which agents to invoke and in what order. The available agents cover the full standard preprocessing workflow: HTML stripping, language detection, translation, spell correction, sentence segmentation, tokenization, named entity recognition (NER), and Pydantic schema validation. The LLM is never permitted to perform any transformation itself; its role is strictly classificatory, ensuring that the system degrades gracefully if the model is slow or unavailable and that every transformation output remains deterministic and auditable.

On the image side, a rule-based decision engine evaluates three quantitative quality metrics — Laplacian variance for blur, mean luminance for brightness, and pixel-distribution standard deviation for contrast — and selects only the agents a given image actually requires: deblurring via Wiener deconvolution if the image is out of focus, contrast and brightness enhancement via CLAHE if exposure is outside acceptable bounds, Non-Local Means denoising if the image is low-contrast and noisy, and region-of-interest cropping as a near-universal preprocessing step. The deliberate choice of a rule-based engine over an LLM for image decisions reflects the fundamentally quantitative nature of image quality assessment, where scalar thresholds provide fully reproducible, zero-latency decisions that an LLM cannot improve upon.

Both pipelines incorporate post-transformation validation as a safety net. Text outputs are validated against a typed Pydantic schema, catching structural errors before records are published downstream. Image transformations are accepted or automatically reverted based on PSNR and SSIM thresholds — SSIM must be at least 0.995 and PSNR at least 15.0 dB — ensuring that no processing step silently degrades the signal it was intended to improve. The entire system is unified under an Apache Kafka streaming backbone that decouples ingestion, type-based routing, processing, and persistence across five topics, enabling row-level parallelism for large text files and fully asynchronous image processing without blocking the API layer. Results are persisted to MongoDB and streamed to a React-based monitoring frontend via Server-Sent Events.

Empirical evaluation demonstrated that selective text execution achieved a mean latency reduction of 35 to 40 percent relative to a fixed full-pipeline approach applying all eight agents unconditionally. For individual input categories, reductions of up to 87 percent were observed. On the image side, the quality-aware selective approach achieved a 92 percent post-transformation validation acceptance rate, compared to 68 percent for a blind fixed-sequence baseline — a 24 percentage point improvement. All stated design targets for cleaning success rate, processing latency, throughput, and error rate were met or exceeded. The framework demonstrates that combining LLM-guided planning for semantically complex decisions with quantitative rule-based planning for measurable quality decisions, unified under a streaming infrastructure with per-step validation and revert capability, produces a demonstrably more adaptive, resource-efficient, and auditable alternative to static preprocessing pipelines.

Keywords: agentic AI, LLM orchestration, multi-modal data preprocessing, image quality assessment, Kafka streaming, PSNR, SSIM, named entity recognition, adaptive pipeline, Wiener deconvolution, CLAHE, NLMeans

TABLE OF CONTENTS

01. INTRODUCTION	12
1.1 Background.....	12
1.2 Problem Statement	13
1.3 Research Scope	14
1.4 Related Work	14
1.4.1 Agentic AI and LLM Orchestration.....	14
1.4.2 Text Processing Pipelines	15
1.4.3 Image Quality Assessment and Enhancement	16
1.4.4 Streaming Architectures for ML Pipelines	17
1.5 Research Gap	18
1.6 Research Questions	18
1.7 Objectives	19
1.7.1 Aim	19
1.7.2 Main Objective.....	19
1.7.3 Sub-Objectives	19
02. METHODOLOGY	21
2.1 System Architecture.....	21
2.2 Technology Stack.....	22
2.3 LLM-Guided Agent Planning	23
2.3.1 Prompt Engineering and Canonicalization.....	23
2.3.2 Agent-Level Decision Logic	24
2.4 Text Agents.....	25
2.4.1 HTML Strip Agent.....	25
2.4.2 Language Detection Agent.....	25
2.4.3 Translator Agent	25
2.4.4 Spell Correction Agent.....	26
2.4.5 Sentence Splitter and Tokenizer Agents	26
2.4.6 NER Agent.....	26
2.4.7 Validation Agent.....	27
2.5 Image Quality Metric Computation	28
2.5.1 Blur Metric.....	28
2.5.2 Brightness Metric	28
2.5.3 Contrast Metric	29

2.6 Rule-Based Decision Engine	29
2.7 Image Agents	30
2.7.1 ROI Agent.....	30
2.7.2 Enhancement Agent	30
2.7.3 Denoise Agent.....	31
2.7.4 Deblur Agent.....	31
2.8 Post-Transformation Quality Validation.....	32
2.9 Text Pipeline Problem Formulation.....	33
2.10 Image Orchestration and Validation Protocol.....	33
2.11 Persistence and Real-Time Monitoring.....	34
03. RESULTS AND DISCUSSION	35
3.1 Text Processing Performance.....	35
3.1.1 Cleaning Success Rate and Confidence	35
3.1.2 Agent Utilization Rates	36
3.1.3 Latency Reduction	36
3.2 Image Processing Performance.....	37
3.2.1 Transformation Acceptance Rate	37
3.2.2 Per-Agent Quality Metrics	37
3.2.3 Quality-Aware vs Blind Processing.....	38
3.3 System-Level Performance	38
3.4 Discussion	39
3.4.1 Interpretation of Core Findings.....	39
3.4.2 Asymmetry Between Text and Image Decision Mechanisms	40
3.4.3 Architectural Implications of Kafka-Based Decoupling.....	41
3.4.4 Broader Applicability	41
3.4.5 Limitations and Honest Acknowledgements	42
04. CONCLUSION.....	43
4.1 Conclusions.....	43
4.2 Recommendations and Future Work.....	44
REFERENCES	46

LIST OF TABLES

Table 1Kafka Topic Pipeline	22
Table 2Agent Utilization Rates by Input Category.....	36
Table 3Latency Reduction by Input Category	36
Table 4Per-Agent Image Quality Metrics (Mean over Accepted Transformations).....	37
Table 5Blind vs Quality-Aware Processing Comparison	38
Table 6System-Level Performance Summary	38

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CLAHE	Contrast Limited Adaptive Histogram Equalization
CSV	Comma-Separated Values
FFT	Fast Fourier Transform
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ITU-R	International Telecommunication Union — Radiocommunication Sector
JSON	JavaScript Object Notation
JSONL	JSON Lines (newline-delimited JSON)
LLM	Large Language Model
MSE	Mean Squared Error
NER	Named Entity Recognition
NLTK	Natural Language Toolkit
NLMeans	Non-Local Means Denoising Algorithm
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PSNR	Peak Signal-to-Noise Ratio (dB)
PSF	Point Spread Function
RAM	Random Access Memory
REST	Representational State Transfer
ROI	Region of Interest
SSE	Server-Sent Events
SSIM	Structural Similarity Index Measure
TXT	Plain Text File Format
UUID	Universally Unique Identifier

01. INTRODUCTION

1.1 Background

Every supervised learning system begins with data preparation. The quality, consistency, and appropriateness of that preparation directly shapes not only the accuracy of any model trained on the resulting data but also the reproducibility of experiments, the fairness of predictions, and the operational cost of the pipeline in production. Despite the centrality of this step, the overwhelming majority of production systems still rely on fixed, hand-crafted preprocessing sequences: the same operations are executed in the same order for every input record, regardless of whether that particular record requires them.

This uniformity has deep historical roots. When datasets were small, homogeneous, and curated, a fixed pipeline was a reasonable approximation. A dataset of English news articles from a single source would uniformly benefit from tokenization and stopword removal. A dataset of photographs taken under controlled laboratory conditions would uniformly benefit from histogram equalization. In such settings, the cost of applying unnecessary operations was low, and the risk of applying harmful operations was manageable because data variability was limited.

The modern data landscape looks nothing like this. Organizations now ingest data from dozens of sources simultaneously: web-scraped text in multiple languages and with varying degrees of HTML contamination, images from cameras with heterogeneous sensor characteristics and lighting conditions, CSV files exported from legacy systems with inconsistent encoding, and real-time streams from IoT devices operating under unpredictable environmental conditions. In this environment, a fixed pipeline is not merely suboptimal — it is actively harmful. Applying HTML stripping to a dataset that contains no HTML wastes compute time and risks altering edge cases such as angle brackets in mathematical notation. Applying spell correction to text that is already clean risks inadvertent substitutions of domain-specific terminology. Applying contrast enhancement to an image that is already well-exposed introduces over-amplification and compression artifacts.

The emergence of agentic AI architectures offers a principled solution. Rather than encoding preprocessing logic as a static script, an agentic system can inspect its input, reason about what that input requires, and dynamically assemble a tailored sequence of operations. This approach draws on a growing body of work in LLM-guided task planning — from the ReAct framework, which established the pattern of interleaving reasoning traces with action execution in LLM-based agents, to HuggingGPT, which extended this idea to multi-modal settings by using ChatGPT as a planner to route tasks to specialist models. Crucially, however,

these frameworks have primarily been applied to high-level task completion — answering questions, generating content, calling APIs — rather than to the precise, measurable domain of data preprocessing, where the inputs are structured, the operations are well-defined, and the correctness of each transformation can be verified objectively.

This dissertation describes an agentic multi-modal preprocessing framework that applies these architectural ideas specifically to the preprocessing domain. The framework handles both text and images through a unified streaming infrastructure and demonstrates that combining LLM-guided planning with quantitative quality assessment and per-step validation produces measurable improvements in both latency and output quality relative to static baselines.

1.2 Problem Statement

The fundamental problem addressed in this dissertation can be stated precisely. Given a stream of heterogeneous input records — where inputs may be text strings in arbitrary languages with varying degrees of markup and spelling noise, or images with variable blur, brightness, and contrast characteristics — the goal is to apply exactly the set of preprocessing transformations that each record requires, in the correct order, without applying transformations that are unnecessary or that would degrade already-acceptable signal.

This problem has three distinct subproblems that existing systems address inadequately. The first is the agent selection problem: given an input, how should the system determine which preprocessing operations it requires? Existing systems solve this by running all operations unconditionally, which is wasteful and risky. A system that solves this subproblem adaptively must be capable of inspecting the input and making an informed decision before any transformation is applied.

The second is the quality assurance problem: given that an agent has been selected and has produced a transformation, how should the system verify that the transformation has improved rather than degraded the input? Existing systems do not address this at all — they assume that applying a preprocessing step always improves the data. This assumption is violated whenever an agent is applied to data that did not require it, or whenever an agent's parameter settings are poorly matched to the characteristics of a particular input.

The third is the infrastructure problem: how should the system manage the ingestion, routing, processing, and persistence of a continuous stream of heterogeneous records in a way that is asynchronous, parallelizable, and fault-tolerant? Existing preprocessing systems are typically synchronous batch

processors that block on each record until processing is complete, making them unsuitable for high-velocity streaming environments.

This framework addresses all three subproblems through an integrated architecture that combines LLM-guided planning, quantitative quality validation with automatic revert capability, and an Apache Kafka streaming backbone.

1.3 Research Scope

The scope of this dissertation is the design, implementation, and empirical evaluation of an agentic multi-modal data preprocessing framework that covers the following components:

- A text preprocessing pipeline orchestrated by a locally-hosted LLM (Ollama) that dynamically selects and sequences agents from a catalogue of eight deterministic text-processing functions.
- An image preprocessing pipeline orchestrated by a rule-based decision engine that evaluates three quantitative quality metrics and selects from a catalogue of four image processing agents.
- A post-transformation quality validation layer that uses PSNR and SSIM to evaluate image transformations and automatically reverts those that do not meet acceptance thresholds.
- A Pydantic schema validation layer that verifies text processing outputs against a typed record model before downstream publication.
- An Apache Kafka streaming backbone comprising five topics that decouple ingestion, routing, text processing, image processing, and persistence.
- A MongoDB persistence layer and a React-based real-time monitoring frontend connected via Server-Sent Events.

The scope explicitly excludes the training or fine-tuning of any machine learning models. The LLM used for text planning is a pre-trained model accessed via the Ollama inference server. The image processing agents are implemented using classical computer vision algorithms. The NER capability is provided by the pre-trained spaCy `en_core_web_sm` pipeline.

1.4 Related Work

1.4.1 Agentic AI and LLM Orchestration

The concept of using language models as orchestrators of specialist tools rather than as generators of final outputs was crystallized by the ReAct framework (Yao et al., 2023). ReAct demonstrated that LLMs can interleave reasoning traces — expressed in natural language — with discrete action calls to external tools, creating a closed loop in which the model can observe the results of its actions and refine its plan accordingly. This observation-planning-action loop is the foundational architectural pattern that the OllamaPlanner in this work instantiates for the specific domain of text preprocessing.

HuggingGPT (Shen et al., 2023) extended this idea to multi-modal settings by deploying ChatGPT as a central planner that routes tasks to specialist models hosted on the HuggingFace model hub. The system

demonstrated that a single planning LLM could coordinate image captioning, object detection, text generation, and speech processing tasks by selecting the appropriate specialist model for each subtask. The multi-modal preprocessing framework described in this dissertation shares this planner-plus-specialists structure but applies it to a fundamentally different problem: rather than routing tasks to generative models, it routes data records to deterministic transformation functions, and rather than accepting the planner's output as a final product, it validates every transformation output before committing it.

The taxonomy of agentic AI architectures provided by Gioacchini et al. (2026) categorizes systems along several dimensions: single-agent versus multi-agent, tool-augmented versus autonomous, and reactive versus deliberative. The text processing pipeline in this framework occupies the tool-augmented single-planner category — the LLM does not act autonomously but selects from a fixed, pre-defined catalogue of tools. The image processing pipeline occupies a fully rule-based category, reflecting the decision that quantitative image quality decisions do not benefit from the open-ended reasoning that makes LLMs valuable for text analysis. This deliberate asymmetry is a key design contribution of the framework.

1.4.2 Text Processing Pipelines

The standard toolkit for NLP preprocessing has been relatively stable for over a decade. NLTK (Bird et al., 2009) introduced the Punkt sentence tokenizer, which uses an unsupervised algorithm to learn collocations for abbreviations and contractions and apply them to sentence boundary detection. This remains the state of the art for language-agnostic sentence segmentation in the absence of a transformer-based segmenter, and it handles the edge cases — periods after abbreviations, decimal numbers, ellipses — that naive rule-based approaches fail on.

spaCy (Honnibal et al., 2020) extended this toolkit with a production-grade NLP pipeline that includes tokenization, part-of-speech tagging, dependency parsing, and named entity recognition. The NER component, backed by a transformer-based architecture in its larger variants, identifies entities of types including PERSON, ORG, GPE, PRODUCT, DATE, TIME, LAW, and EVENT across a broad range of text types and genres. Language identification is handled by `langid.py` (Lui and Baldwin, 2012), a compact library that uses a Naive Bayes classifier trained on character n-gram features extracted from text in 97 languages, achieving strong identification accuracy with minimal computational overhead — a critical property for a component that must run on every record before any other processing.

Spell correction libraries such as `pyspellchecker` operate on dictionary-lookup principles, computing the ratio of out-of-dictionary tokens to total token count as a proxy for text noisiness. A critical design question is when to apply spell correction and when to suppress it: correction is beneficial for genuinely misspelled words but harmful for domain-specific terminology, proper nouns, and deliberate non-standard usage. The

framework addresses this through the noise ratio threshold mechanism, suppressing correction for texts where fewer than 10 percent of tokens are out-of-dictionary.

Multilingual text introduces additional complexity. The Google Cloud Translation API provides neural machine translation between hundreds of language pairs, enabling the pipeline to normalize non-English inputs to English before applying English-specific agents such as spell correction and NER. The dynamic insertion of the translator agent by the execution engine — rather than requiring the LLM to specify it explicitly — encodes the hard dependency between language detection and translation in the pipeline infrastructure itself, making it robust to LLM planning errors.

1.4.3 Image Quality Assessment and Enhancement

No-reference image quality assessment — the problem of evaluating image quality without access to a ground truth original — is a well-studied problem with a large literature. Pertuz et al. (2013) conducted the most comprehensive systematic comparison to date, evaluating 36 focus measure operators across datasets of images with controlled blur levels. Their analysis established the Laplacian variance as one of the most reliable and computationally efficient operators in this class, providing the empirical grounding for its use as the blur quality metric in this framework.

The Structural Similarity Index Measure (SSIM) was introduced by Wang et al. (2004) as a perceptually motivated alternative to PSNR for full-reference image quality assessment. The key insight of SSIM is that human perception of image quality is driven more by structural information — the relationships between adjacent pixels — than by pixel-wise differences, which PSNR measures. SSIM decomposes image quality into three components: luminance similarity, contrast similarity, and structural similarity, and combines them into a single scalar in the range $[-1, 1]$, where 1 indicates identical images. The use of SSIM alongside PSNR in the post-transformation validation layer provides both a perceptually grounded measure of structural preservation and an absolute measure of signal fidelity.

Contrast enhancement through CLAHE was proposed by Zuiderveld (1994) as an improvement over global histogram equalization. The key limitation of global equalization is that it treats the entire image as a single distribution, which can dramatically over-amplify noise in dark uniform regions while compressing dynamic range in bright regions. CLAHE addresses this by applying histogram equalization locally within non-overlapping tiles and introducing a clip limit that bounds the amplification rate of any histogram bin. The LAB color space is used for enhancement rather than RGB because the L channel captures luminance independently of chrominance, allowing equalization to be applied to brightness without shifting hue or saturation.

The Non-Local Means (NLMeans) denoising algorithm, introduced by Buades et al. (2005), exploits the self-similarity of natural images at the patch level. For each pixel, NLMeans computes a weighted average over all pixels in the image whose local neighborhoods are similar to the neighborhood of the target pixel, regardless of spatial proximity. This global search allows the algorithm to recover fine texture details that locally-averaging filters such as Gaussian blur or bilateral filtering destroy. The bilateral filter (Tomasi and Manduchi, 1998) is retained as a fallback for computational efficiency in environments where NLMeans is too slow.

Wiener deconvolution (Orieux et al., 2010) addresses blur caused by a known or estimated point-spread function by applying the Wiener filter in the frequency domain. The filter simultaneously inverts the blurring effect of the PSF and suppresses amplification of noise at frequencies where the PSF response is weak. The regularization parameter λ controls the noise-sharpness trade-off: a small λ produces aggressive sharpening with noise amplification, while a large λ produces conservative denoising at the cost of incomplete restoration.

1.4.4 Streaming Architectures for ML Pipelines

Apache Kafka (Kreps et al., 2011) was originally designed as a distributed log system for log aggregation and metrics collection at LinkedIn. Its core abstraction — a distributed commit log with configurable retention and consumer group semantics — proved so broadly applicable that it has since become the de facto standard for event streaming in large-scale data architectures. The consumer group model allows multiple consumer instances to share the load of processing a single topic's partitions, providing horizontal scalability without modifications to either the producer or the consumer logic.

In the context of ML preprocessing pipelines, Kafka's properties offer three specific advantages over synchronous batch processing. First, decoupling: by publishing raw inputs to a topic and consuming processed outputs from a separate topic, the ingestion layer is completely isolated from the processing layer, allowing each to scale independently. Second, row-level parallelism: large files can be split into individual rows at ingestion time and published as independent messages, allowing thousands of rows to be processed simultaneously by a consumer group with many instances. Third, fault isolation: a crash in the image processing consumer does not affect the text processing consumer, because they subscribe to separate topics with independent offset tracking.

FastAPI (Ramirez, 2018) provides the REST and SSE API layer. Its native asynchronous request handling via Python's `asyncio` enables concurrent processing of multiple upload requests without the thread-per-request overhead of synchronous frameworks. The `/events` endpoint, implemented as an async generator

that polls an in-memory FIFO buffer and emits newly-seen records as JSON over the SSE protocol, provides real-time monitoring of pipeline activity without requiring the frontend to poll the database.

1.5 Research Gap

The literature reviewed above reveals three interconnected gaps that no existing system addresses simultaneously.

Gap 1 — Adaptive Agent Selection: Existing preprocessing frameworks, whether implemented as static scripts, DAG-based workflow tools, or cloud-managed ETL services, apply operations uniformly to all inputs. The only adaptive behavior typically implemented is conditional branching based on file type (text versus image), not based on the quality or content of individual records within each type. No published system applies LLM-guided dynamic planning to select text preprocessing agents, nor rule-based quantitative quality assessment to select image preprocessing agents, on a per-record basis.

Gap 2 — Per-Step Quality Validation with Revert: Existing image preprocessing pipelines apply transformations sequentially and commit all outputs regardless of whether each transformation has improved or degraded the input. Post-processing quality assessment is sometimes applied at the end of a pipeline to evaluate the overall output, but per-step validation with automatic revert capability — which prevents a single poorly-matched transformation from degrading all subsequent steps — has not been described in the preprocessing literature.

Gap 3 — Unified Multi-Modal Streaming with Real-Time Monitoring: Text and image preprocessing are typically handled by entirely separate toolchains with no shared infrastructure for ingestion routing, result persistence, or monitoring. A unified streaming architecture that handles both modalities under a single API, routes records to the appropriate processing pipeline, and delivers results to a real-time monitoring frontend without polling does not appear in the existing literature on preprocessing systems.

This framework closes all three gaps through an integrated architecture that is simultaneously adaptive, validating, multi-modal, and streaming-native.

1.6 Research Questions

The following research questions guide the design and evaluation of this framework:

RQ1: How effectively can an LLM-guided planning layer select the appropriate subset of text preprocessing agents for heterogeneous inputs, and what mean latency reduction does selective execution achieve compared to a fixed full-pipeline approach applying all eight agents unconditionally?

RQ2: Can a rule-based decision engine driven by three quantitative image quality metrics — Laplacian variance for blur, mean luminance for brightness, and pixel standard deviation for contrast — reliably select the correct image preprocessing agents across diverse input quality conditions?

RQ3: Does per-step PSNR/SSIM validation with automatic revert capability produce a statistically meaningful improvement in image transformation acceptance rate compared to a blind fixed-sequence baseline?

RQ4: Can an Apache Kafka-based streaming backbone decouple ingestion, routing, processing, and persistence across text and image modalities sufficiently to enable asynchronous, parallelized preprocessing with sub-3-second text latency and sub-10-second image latency at the design target throughput rates?

RQ5: Does the deliberate asymmetry between LLM-based text planning and rule-based image planning produce better overall outcomes than a uniform approach to both modalities, and what does this asymmetry imply about the appropriate use of LLMs in agentic preprocessing systems?

1.7 Objectives

1.7.1 Aim

To design, implement, and empirically evaluate an agentic multi-modal data preprocessing framework that achieves input-adaptive agent selection, per-step quality validation, and real-time streaming delivery across text and image modalities.

1.7.2 Main Objective

To develop and deploy a production-grade agentic preprocessing pipeline in which each incoming record — whether text or image — receives exactly the preprocessing operations it requires based on a dynamic assessment of its content and quality characteristics, with every transformation validated before commitment and with all activity streamed in real time to a monitoring frontend.

1.7.3 Sub-Objectives

- To conduct a comprehensive review of agentic AI orchestration frameworks, standard NLP preprocessing toolkits, image quality assessment and enhancement algorithms, and streaming infrastructure for ML pipelines, in order to identify best-in-class components for each layer of the proposed architecture.
- To design and implement an LLM-guided text preprocessing orchestrator using a locally-hosted Ollama model, a prompt engineering approach that produces JSON agent plans, a canonicalization mechanism that enforces logical execution dependencies, and a conservative fallback that activates when the LLM is unavailable.
- To design and implement eight deterministic text preprocessing agents — HTML stripping, language detection, translation, spell correction, sentence segmentation, tokenization, NER, and Pydantic validation — each with self-evaluating confidence scores and precondition flags.

- To design and implement a rule-based image preprocessing decision engine that computes Laplacian variance, mean luminance, and contrast standard deviation and applies threshold logic to produce an ordered action list with configurable parameters.
- To design and implement four image preprocessing agents — ROI cropping, contrast/brightness enhancement via CLAHE, NLMeans denoising, and Wiener deconvolution-based deblurring — each implementing a fallback for environments without OpenCV GPU support.
- To implement a post-transformation quality validator that computes PSNR and SSIM between before and after images, enforces acceptance thresholds of $SSIM \geq 0.995$ and $PSNR \geq 15.0$ dB, and automatically reverts rejected transformations.
- To build a five-topic Apache Kafka streaming backbone that handles raw input ingestion, type-based routing, text processing, image processing, and unified result persistence.
- To implement a MongoDB persistence layer with UUID-based document identity and a FastAPI SSE endpoint that delivers processed records to a React monitoring frontend in real time.
- To empirically evaluate the complete framework on a representative corpus of text and image inputs, measuring text cleaning success rate, agent utilization rates, latency reduction by input category, image transformation acceptance rate, PSNR and SSIM distributions by agent, and overall system-level throughput and latency.

02. METHODOLOGY

2.1 System Architecture

The framework is organized into four functional layers that operate in sequence for every input record: a REST/SSE API layer responsible for ingestion and real-time delivery, a Kafka-based message streaming layer responsible for type detection, routing, and inter-component communication, a processing layer containing the `AgenticTextProcessor` and the `ImagePreprocOrchestrator`, and a persistence layer backed by MongoDB. The layered architecture ensures that each concern is handled by a single, independently deployable component, and that adding new modalities or agents does not require changes to the infrastructure layers.

Inputs arrive at the `/ingest` endpoint as one of three types: plain text strings, delimited text files (CSV, JSONL, TXT), or image files (JPEG, PNG, TIFF, and other formats recognized by magic number inspection). The API layer applies an initial type classification before enqueueing: text strings are published directly as JSON messages with a `text` key; text files are split row by row at ingestion time, with each row published as an independent message; image files are written to disk under a request-specific directory and their path reference is enqueued as a JSON message with an `image_path` key, avoiding Kafka's default message size limit of 1 MB for binary content.

A router worker subscribes to the `raw_input` topic and performs secondary type inspection on each message. It examines the presence of `text`, `image`, or `image_path` keys in the JSON payload, or applies magic number detection to raw bytes that may have bypassed JSON encoding. Verified text payloads are forwarded to the `detected_text` topic; verified image references are forwarded to the `detected_image` topic. This two-stage typing — once at the API layer and once at the router — ensures that ambiguous or malformed payloads are caught before they reach the processing layer.

Separate Kafka consumer threads subscribe to `detected_text` and `detected_image` respectively. The text consumer invokes the `AgenticTextProcessor` on each payload, which internally manages the full LLM planning and agent execution loop. The image consumer loads the image from disk, decodes it into an RGB NumPy array, and passes it to the `ImagePreprocOrchestrator`, which manages quality metric computation, agent selection, agent execution, and per-step validation. Both processors publish their results to `processed_text` and `processed_image` topics respectively.

A unified results worker subscribes to both processed topics. It normalizes the record structure — which differs between text and image outputs — into a common schema with shared fields for request UUID, input type, timestamp, processing duration, and a modality-specific payload field. The normalized record

is upserted into MongoDB using the UUID as the document `_id`, ensuring idempotent processing. Simultaneously, the record is appended to a thread-safe in-memory FIFO buffer capped at 200 entries, which feeds the SSE stream consumed by the React frontend.

Table 1 summarizes the five Kafka topics, their producers, and their consumers.

Table 1Kafka Topic Pipeline

Topic	Producer	Consumer
raw_input	POST /ingest endpoint	Router worker
detected_text	Router worker	Text consumer thread
detected_image	Router worker	Image consumer thread
processed_text	AgenticTextProcessor	Results worker
processed_image	ImagePreprocOrchestrator	Results worker

2.2 Technology Stack

The backend is implemented in Python 3.11 using FastAPI as the asynchronous web framework. FastAPI's native asyncio integration allows the `/ingest` endpoint to accept uploads concurrently without thread-per-request overhead, and its automatic OpenAPI documentation generation facilitates integration testing. Apache Kafka is deployed as a single-broker cluster for evaluation purposes, with the `kafka-python` client library providing consumer and producer interfaces.

The LLM planning component uses the LangChain Ollama integration to communicate with a locally hosted Llama 3 8B model served by the Ollama inference server. The locally hosted deployment was chosen deliberately to avoid API latency variability, to maintain data locality, and to enable deployment in air-gapped environments where cloud API access is unavailable. The image processing layer relies on OpenCV for image I/O, color space conversion, edge detection, and Gaussian operations, with NumPy and `scipy.ndimage` providing fallback implementations for environments where OpenCV is unavailable.

The NLP stack uses spaCy's `en_core_web_sm` pipeline for NER, NLTK for sentence tokenization and word tokenization, `langid.py` for language identification, `pyspellchecker` for spell correction, and the Google Cloud Translation API for machine translation. Pydantic v2 provides schema validation. MongoDB is accessed via the `pymongo` driver, and Server-Sent Events are implemented using FastAPI's `StreamingResponse` with an async generator. The React frontend is built with React 18, TypeScript, and Chart.js for the results visualization components.

2.3 LLM-Guided Agent Planning

The `AgenticTextProcessor` delegates the agent selection decision entirely to the `OllamaPlanner` class, which encapsulates all LLM interaction behind a clean interface that returns an ordered list of agent names. This encapsulation ensures that the rest of the text processing pipeline is indifferent to whether the plan was produced by an LLM, a fallback rule, or any future replacement mechanism.

The `OllamaPlanner` is instantiated with a reference to the locally-hosted Ollama model, a timeout parameter that bounds the maximum planning latency, and a list of the eight available agent identifiers. For each incoming text sample, the planner invokes its `plan()` method, which constructs a prompt by injecting the raw input text into a fixed instruction template and submitting it to the model via the LangChain Ollama interface.

The eight available agent identifiers are: `html_strip_agent`, `lang_detect_agent`, `translator_agent`, `spell_agent`, `sentence_split_agent`, `tokenizer_agent`, `ner_agent`, and `validation_agent`. The instruction template presents these identifiers as a fixed menu and instructs the model to return a JSON array of the agent names it believes should be applied to the given input, with no additional commentary or explanation.

2.3.1 Prompt Engineering and Canonicalization

The prompt design follows three principles. First, the output format is constrained to a JSON array of string identifiers, minimizing the parsing burden and reducing the surface area for malformed responses. Second, the prompt explicitly instructs the model not to include agents that are not needed for the given input, rather than asking it to include all agents that might be useful — this framing encourages conservative selection, which aligns with the goal of reducing unnecessary execution. Third, the prompt includes a brief description of each agent's purpose, which improves selection accuracy for agents whose names are not self-explanatory.

The model's response is parsed as JSON and filtered to retain only valid agent identifiers. The resulting subset is then reordered into a canonical execution sequence that preserves logical dependencies regardless of the order in which the model listed agents. The canonical ordering is: `[html_strip_agent, lang_detect_agent, translator_agent, spell_agent, sentence_split_agent, tokenizer_agent, ner_agent, validation_agent]`. The relative order of any subset selected by the LLM is preserved within this canonical sequence.

This canonicalization step is essential for correctness. The LLM cannot reliably be expected to always output agents in the correct dependency order: it might list `ner_agent` before `lang_detect_agent`, or `spell_agent` before `translator_agent`. The canonical ordering resolves these inconsistencies deterministically, ensuring that the execution engine always receives a valid, dependency-respecting plan.

If the LLM call times out or returns malformed JSON that cannot be parsed as a list of agent identifiers, a conservative fallback plan activates: [spell_agent, tokenizer_agent, lang_detect_agent]. These three agents cover the most common preprocessing needs — noise reduction, token extraction, and language identification — without requiring any agents that carry significant risk of inadvertent modification, such as the translator or the HTML stripper.

2.3.2 Agent-Level Decision Logic

Even when the LLM includes an agent in its plan, each agent applies its own internal precondition check before executing. This agent-level gating provides a second layer of protection against unnecessary or harmful execution, complementing the LLM's plan-level selection.

The HTML Strip Agent scans the input for patterns matching the regex `<.*?>` and skips execution if no matches are found, returning a confidence of 1.0 and the input unchanged. The Language Detection Agent skips inputs shorter than three characters or containing no alphabetic content, labelling them unknown. The Spell Correction Agent computes the noise ratio as the count of out-of-dictionary tokens divided by the total token count; if this ratio falls below 0.10, the agent returns the original text, reasoning that the correction risk outweighs the benefit for near-clean inputs. The NER Agent requires both that the detected language is English — to ensure that the spaCy English model is appropriate for the input — and that the word count is at least three, since the model lacks sufficient context to produce reliable entity labels for shorter inputs.

The Translator Agent introduces a dynamic plan modification at runtime. When the Language Detection Agent identifies a non-English input, the execution engine inserts the Translator Agent into the remaining plan immediately after the current step, before any English-specific agents execute. This insertion is not a re-query to the LLM; it is a rule applied by the execution engine itself, reflecting the hard dependency between language and downstream agent eligibility.

Each agent records three metadata fields alongside its output: a confidence score in $[0, 1]$ quantifying the reliability of its output, a boolean changed flag indicating whether it actually modified its input, and a boolean should_run flag encoding whether its preconditions were satisfied. This metadata is recorded in the audit trail alongside the agent output, making the full pipeline execution transparent and reproducible. The overall record confidence is computed as the arithmetic mean of individual agent confidence scores across all agents that executed.

2.4 Text Agents

2.4.1 HTML Strip Agent

The HTML Strip Agent applies a two-pass regex over the input string. The first pass matches all substrings conforming to the pattern `<[<^>]+>`, which captures both opening and closing HTML tags, self-closing tags, and comment blocks. The second pass collapses any residual whitespace sequences — multiple consecutive spaces, tabs, or newlines — into a single space and trims leading and trailing whitespace. The agent sets its changed flag to true only if the first pass produces at least one match; for inputs containing no HTML markup, it returns the input unchanged with a confidence of 1.0 and changed set to false.

The regex approach was chosen over a parser-based approach such as BeautifulSoup for two reasons. First, parser-based approaches require constructing a full HTML parse tree, which is computationally expensive for short text snippets that happen to contain a small number of tags. Second, the framework's goal is not to render or interpret HTML but merely to strip markup characters from text that is otherwise readable prose — a problem for which regex is both sufficient and fast.

2.4.2 Language Detection Agent

Language detection uses the `langid.py` library, which implements a Naive Bayes classifier trained on character n-gram features extracted from text in 97 languages. The classifier operates entirely at the character level, making it robust to technical terminology and proper nouns that might confuse word-level classifiers. At inference time, `langid.py` computes log-probability scores for each language and returns the most probable language identifier along with its log-probability as a proxy for detection confidence.

The agent applies two precondition checks before invoking the classifier: the input must be at least three characters long, and it must contain at least one alphabetic character. Inputs that fail either check are labelled unknown with a confidence of 0.5, indicating that the language cannot be reliably determined. For inputs that pass both checks, the agent invokes `langid.py` and normalizes the returned log-probability to a confidence score in $[0, 1]$ using a sigmoid transformation.

2.4.3 Translator Agent

When a non-English language is detected, the Translator Agent queries the Google Cloud Translation API to convert the input to English. The API request includes the detected source language code, which improves translation quality by avoiding the source language auto-detection step that the API would otherwise perform. The translated text replaces the input for all subsequent agents in the execution plan.

The translator agent assigns a confidence score of 0.9 for supported language pairs and 0.7 for language pairs that the API detects as low-confidence. If the API call fails — due to network unavailability, quota exhaustion, or an unsupported language pair — the agent returns the original text unchanged, sets its

changed flag to false, and records a confidence of 0.3 to signal degraded reliability to the record confidence aggregator.

2.4.4 Spell Correction Agent

The Spell Correction Agent uses `pyspellchecker` to identify and correct misspelled tokens. The agent first tokenizes the input by splitting on whitespace and punctuation, retaining only alphabetic tokens for dictionary lookup. It computes the noise ratio as the fraction of these tokens that are not found in `pyspellchecker`'s dictionary. If the noise ratio is below 0.10, the agent returns the original text unchanged — the correction risk for near-clean inputs outweighs the benefit, since `pyspellchecker` will occasionally suggest incorrect corrections for valid uncommon words.

Above the threshold, each out-of-dictionary alphabetic token is individually corrected by querying `pyspellchecker` for the most probable correction based on edit distance. Tokens that `pyspellchecker` cannot correct are left unchanged. Punctuation, numbers, and mixed-case tokens that contain uppercase characters beyond the first position (likely proper nouns or acronyms) are excluded from correction to reduce false positives.

2.4.5 Sentence Splitter and Tokenizer Agents

The Sentence Splitter Agent wraps NLTK's `sent_tokenize` function, which uses the Punkt unsupervised sentence boundary detector. Punkt learns a set of abbreviations and collocations from the input text itself before applying boundary detection, making it more robust than rule-based period-plus-space heuristics for texts containing abbreviations such as 'Dr.', 'e.g.', or 'U.S.' that would otherwise trigger false sentence boundaries. The agent returns a list of sentence strings as part of the output record.

The Tokenizer Agent wraps NLTK's `word_tokenize` function, which applies a combination of the `TreebankWordTokenizer` and `MWETokenizer` to handle contractions, hyphenated compounds, and punctuation correctly. Unlike simple whitespace splitting, `word_tokenize` correctly separates 'isn't' into ['is', 'n't'], separates 'end.' into ['end', '.'], and handles a range of punctuation scenarios that simpler approaches fail on. The agent returns a list of token strings as part of the output record.

2.4.6 NER Agent

The Named Entity Recognition Agent passes the processed English text to `spaCy`'s `en_core_web_sm` pipeline. This pipeline applies tokenization, part-of-speech tagging, dependency parsing, and entity recognition in sequence, returning a Doc object from which named entities can be extracted. The agent extracts entities of the following types: PERSON (people, including fictional), ORG (companies, agencies, institutions), GPE (geopolitical entities — countries, cities, states), PRODUCT (objects, vehicles, foods),

DATE (absolute or relative dates and periods), TIME (times smaller than a day), LAW (named legal documents), and EVENT (named hurricanes, battles, wars, sports events).

The agent applies two precondition checks: the detected language must be English, and the word count must be at least three. Both checks must be satisfied for NER to execute. If either fails, the agent returns an empty entity list, sets its changed flag to false, and records a confidence of 0.5 to indicate that NER was skipped due to precondition failure.

2.4.7 Validation Agent

The Validation Agent is always the final agent in any execution plan. It uses Pydantic v2 to validate the assembled output record against a typed CleanedTextRecord model that specifies required fields and their types: cleaned_text (str), detected_language (str), sentences (list[str] | None), tokens (list[str] | None), entities (list[EntityRecord] | None), and overall_confidence (float in [0, 1]). If validation passes, the agent publishes the validated record to the processed_text Kafka topic. If validation fails due to missing required fields or type mismatches, the agent logs the validation error and publishes a partial record with a validation_failed flag set to true, allowing downstream systems to filter incomplete records.

2.5 Image Quality Metric Computation

The RuleBasedDecisionEngine begins by converting the input RGB image to a single-channel grayscale representation using the ITU-R BT.601 luminance formula, which weights the red, green, and blue channels by their perceptual contribution to luminance: $\text{Gray}(i,j) = 0.299R + 0.587G + 0.114B$. This formula is the industry standard for RGB-to-grayscale conversion in applications where perceptual accuracy is required, and it is used consistently across all three quality metrics.

2.5.1 Blur Metric

Blur is quantified as the variance of the Laplacian response of the grayscale image. The Laplacian operator $\nabla^2 I$ computes the sum of second-order partial derivatives in both spatial dimensions, effectively measuring the rate of change of local gradient magnitude. For a sharp image, edges are crisp and high-contrast, producing a wide distribution of Laplacian response values and a correspondingly high variance. For a blurred image, high-frequency content has been suppressed, edges are soft and low-contrast, and the Laplacian response compresses toward zero, producing a low variance.

Formally: $\text{Blur Score} = \text{Var}(\nabla^2 I)$, where $\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$. The implementation applies the Laplacian kernel $[0, -1, 0; -1, 4, -1; 0, -1, 0]$ via OpenCV's Laplacian function when available, falling back to a NumPy finite-difference approximation that computes the same kernel convolution. The use of variance rather than the raw Laplacian response integrates the sharpness information over the full image, producing a scalar that is stable across images of different sizes and content types.

2.5.2 Brightness Metric

Brightness is defined as the arithmetic mean of the grayscale pixel values across the entire image: $B = (1/HW) \sum \text{Gray}(i,j)$, where H and W are the image height and width in pixels. This provides a global estimate of the image's overall exposure level. A mean below 70 indicates underexposure, in which dark regions dominate and fine detail in shadow areas is obscured. A mean above 200 indicates overexposure, in which bright regions are saturated and highlight detail is lost. Both conditions warrant enhancement.

The brightness metric is intentionally simple — a single global mean — rather than a more sophisticated measure such as the mean luminance of the brightest or darkest quartile. This simplicity was chosen for interpretability and configurability: the thresholds of 70 and 200 have direct, intuitive meanings in terms of pixel value on the 0-255 scale, making them easy to tune for domain-specific deployments.

2.5.3 Contrast Metric

Contrast is defined as the standard deviation of the grayscale pixel distribution: $C = \sqrt{\frac{1}{HW} \sum (\text{Gray}(i,j) - B)^2}$. A low standard deviation indicates that pixel values cluster tightly around the mean — the image appears flat, with little differentiation between regions. This condition is visually uninformative and is frequently associated with noise in compressed or low-light images, where quantization artifacts reduce dynamic range. A high standard deviation indicates a wide spread of pixel values, corresponding to a high-contrast image with clear differentiation between bright and dark regions.

The contrast metric interacts with the decision engine in a compound way: a low contrast score triggers the addition of both the enhancement agent and the denoise agent to the action list. This reflects the empirical observation that low contrast frequently co-occurs with noise, since both conditions arise from insufficient photon collection in low-light imaging scenarios.

2.6 Rule-Based Decision Engine

Given the three scalar quality metrics — blur score, brightness, and contrast — the decision engine applies a threshold logic to produce an ordered action list. The engine begins by unconditionally adding the ROI agent as the first action, since document border removal is broadly beneficial across all image types and carries negligible risk of information loss. The ROI crop is always performed first because subsequent agents operate on the cropped region, reducing their computational cost and avoiding the processing of irrelevant border pixels.

The deblur agent is added when $\text{blur} < 120.0$. This threshold was established by examining the Laplacian variance distribution of a representative set of document scans and natural photographs spanning the range of quality conditions the system is intended to handle. A Laplacian variance below 120.0 corresponds to images where visual blurring is perceptible to a human observer and is likely to impair downstream OCR or feature extraction.

The enhancement agent is added when $\text{brightness} < 70.0$ or $\text{brightness} > 200.0$. The lower bound captures underexposed images in which dark regions dominate; the upper bound captures overexposed images with saturated highlights. Both conditions are addressed by the Enhancement Agent through gamma correction (for underexposure) and CLAHE (for local contrast, applicable to both conditions).

Both the enhancement agent and the denoise agent are added when $\text{contrast} < 60.0$. The enhancement component of this addition addresses the flatness of the image, while the denoise component addresses the noise that frequently accompanies low contrast. Adding both agents ensures that the image is both lifted from its compressed dynamic range and cleaned of the quantization and sensor noise that typically cause it.

After applying these rules, the engine removes duplicate agent references while preserving order, and caps the total action list at three agents. The cap prevents cascading transformations on already-marginal images where multiple deficiencies are detected, ensuring that the pipeline applies a targeted correction rather than a sequence of aggressive transformations that might collectively degrade the image beyond what any single transformation would.

Every decision is logged alongside the raw metric values and normalized badness scores — computed as the distance from each metric's acceptable range divided by the range width — producing a fully explainable audit record that allows administrators to verify the decision engine's behavior on any given input.

2.7 Image Agents

2.7.1 ROI Agent

The Region of Interest Agent extracts the primary content region of the image by applying contour detection to locate the largest connected boundary. The agent first converts the image to grayscale and applies a 5x5 Gaussian blur to suppress high-frequency noise that would produce spurious edge detections. It then applies the Canny edge detector with a lower threshold of 50 and an upper threshold of 150 to produce a binary edge map. External contours are extracted from this edge map using OpenCV's `findContours` function with the `RETR_EXTERNAL` retrieval mode, which returns only the outermost contour of each connected region.

The bounding rectangle of the largest contour by area is computed. If this area exceeds 5 percent of the total image area — a threshold that filters out noise-induced micro-contours — the image is cropped to the bounding rectangle. Images where no contour exceeds the area threshold are returned unchanged, with `changed` set to false. This threshold prevents the agent from aggressively cropping images that genuinely contain content distributed uniformly across the frame.

2.7.2 Enhancement Agent

The Enhancement Agent applies a two-stage process to correct brightness and contrast deficiencies. In the first stage, gamma correction is applied to the input image using the formula $I_{\text{corrected}} = (I / 255)^{\gamma} * 255$, where gamma is set to 0.7 for underexposed images ($\gamma < 1$ brightens the image nonlinearly) and 1.4 for overexposed images ($\gamma > 1$ darkens it). The gamma value is selected based on whether the brightness metric indicated underexposure or overexposure.

In the second stage, Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied to the L channel of the LAB color space representation. The LAB color space separates luminance (L) from color

information (A and B channels), allowing histogram equalization to be applied purely to brightness without shifting hue or saturation. CLAHE is applied with a tile grid size of 8x8 and a clip limit of 2.0. The clip limit bounds the amplification rate of any histogram bin, preventing the noise-brightening effect that global equalization produces in dark uniform regions. The enhanced L channel is merged back with the original A and B channels, and the result is converted back to RGB.

2.7.3 Denoise Agent

The Denoise Agent supports three denoising methods and selects among them based on configuration and availability. The default and primary method is Non-Local Means (NLMeans), which computes a weighted average over all pixels in the image whose local patch neighborhoods are similar to the neighborhood of the target pixel. The patch size, search window, and noise estimate parameters are set to 7x7, 21x21, and 10 respectively, which are the empirically-validated defaults for general-purpose denoising across a range of noise levels.

The bilateral filter is available as a fallback for time-constrained environments. It computes a weighted average over a local spatial window where weights decrease both with spatial distance (as in a Gaussian blur) and with intensity difference. The intensity-difference weighting preserves edge sharpness by reducing the influence of pixels across an edge boundary, making bilateral filtering effective for denoising while preserving texture.

Gaussian blur is available as a third option for the most constrained environments, though it is the weakest denoiser in terms of edge preservation. All three methods are wrapped in the same agent interface, and the method is selectable via configuration parameter without any changes to the orchestration logic.

2.7.4 Deblur Agent

The Deblur Agent implements Wiener deconvolution in the frequency domain. The deconvolution models the blurring process as a convolution of the original sharp image with a point-spread function (PSF), and applies the Wiener filter to invert this convolution while suppressing noise amplification. The PSF is modeled as a uniform box kernel of size $k \times k$, where k is set to 5 by default and is exposed as a configurable parameter. This model holds reasonably well for defocus blur — the dominant blur type in out-of-focus document scans — but is an approximation for other blur types.

The Wiener filter is applied in the frequency domain: $\hat{F}(u,v) = [H^*(u,v) / (|H(u,v)|^2 + \lambda)] \cdot G(u,v)$, where $G(u,v)$ is the FFT of the blurred observation, $H(u,v)$ is the FFT of the PSF, H^* is the complex conjugate of H , and $\lambda = 0.01$ is the regularization parameter. The FFT is computed using NumPy's `fft2` function, and the inverse FFT is applied to produce the restored image. The real part of the complex inverse FFT output is clipped to the $[0, 255]$ range and converted to `uint8`.

When OpenCV is unavailable — which may occur in minimal Python environments — the agent falls back to an unsharp mask: $I_{\text{sharp}} = I + \alpha * (I - \text{GaussianBlur}(I, 0))$, with $\alpha = 1.4$. This approximation increases apparent sharpness by amplifying high-frequency content, though it does not perform true deconvolution and is therefore weaker for severely blurred inputs.

2.8 Post-Transformation Quality Validation

The QualityValidator is invoked after each agent execution in the image pipeline, comparing the state of the image before and after the agent's transformation. If the transformation passes both validation checks, it is committed — the image state is updated to the agent's output and processing continues with the next agent. If either check fails, the agent's output is silently discarded and the pipeline continues from the pre-agent state, as if the agent had not been invoked.

The first validation check is PSNR (Peak Signal-to-Noise Ratio). PSNR is defined as: $\text{PSNR} = 20 * \log_{10}(L / \sqrt{\text{MSE}})$ dB, where $L = 255$ for 8-bit images and MSE is the mean squared per-pixel error between the before and after images: $\text{MSE} = (1/HW) \sum (I_{\text{before}}(i,j) - I_{\text{after}}(i,j))^2$. PSNR provides an absolute measure of signal fidelity: a transformation that introduces significant pixel-level changes produces a low PSNR. The acceptance threshold of 15.0 dB was set to allow the large structural changes introduced by ROI cropping while rejecting transformations that introduce pervasive pixel-level noise.

The second validation check is SSIM (Structural Similarity Index Measure). SSIM captures perceptual image quality by comparing local luminance, contrast, and structural statistics: $\text{SSIM}(x,y) = [(2\mu_x \mu_y + C1)(2\sigma_{xy} + C2)] / [(\mu_x^2 + \mu_y^2 + C1)(\sigma_x^2 + \sigma_y^2 + C2)]$, where μ_x and μ_y are local mean luminance values, σ_x and σ_y are local standard deviations, σ_{xy} is the local cross-correlation, and $C1 = (0.01 * 255)^2$ and $C2 = (0.03 * 255)^2$ are stabilization constants. SSIM is computed using an 11x11 Gaussian-weighted window with sigma 1.5. The acceptance threshold of 0.995 was calibrated to allow ROI cropping (which introduces no within-region distortion) and effective denoising (which reduces pixel variance while preserving structure) while rejecting over-enhancement (which introduces local over-amplification visible to SSIM's structural comparison).

When the output dimensions differ from the input — as they do after ROI cropping — the validator resizes the output to match the input dimensions before computing metrics. This ensures dimensional comparability while correctly measuring the structural fidelity of the retained region. The resize operation uses bilinear interpolation, which introduces a small amount of blurring that marginally reduces SSIM; the threshold of 0.995 was calibrated with this resize step included.

2.9 Text Pipeline Problem Formulation

The text preprocessing problem is formulated as follows. Given a raw text input x drawn from the space S of all Unicode strings, the goal is to produce a cleaned, structured output record y via a sequence of transformations selected dynamically based on the content of x . Each preprocessing agent a_i defines a mapping $a_i: S \rightarrow S$, transforming the current text state. The planner selects an ordered subset $A = [a_1, a_2, \dots, a_k]$ from the full agent catalogue $C = \{\text{html_strip}, \text{lang_detect}, \text{translator}, \text{spell}, \text{sent_split}, \text{tokenizer}, \text{ner}, \text{validation}\}$, and the agents execute sequentially such that the output of each agent becomes the input to the next.

The final record y aggregates the cumulative output of the pipeline, including the cleaned text, detected language, sentence boundaries, token list, and extracted named entities. Each agent a_i is also designed to report whether it modified the input (changed_i), its confidence in its own output (conf_i in $[0,1]$), and whether its preconditions were satisfied (should_run_i). The overall record confidence is $\text{conf}(y) = (1/k) \sum \text{conf}_i$, averaged over all agents that executed.

2.10 Image Orchestration and Validation Protocol

After the decision engine produces an ordered action list $A = [a_1, a_2, \dots, a_k]$, the ImagePreprocOrchestrator executes agents one at a time, applying a validate-then-commit protocol at each step. Before agent a_i runs, the current image state $I_{\{i-1\}}$ is captured as the before reference. After a_i returns its output image $I_{i_candidate}$, the QualityValidator computes $\text{PSNR}(I_{\{i-1\}}, I_{i_candidate})$ and $\text{SSIM}(I_{\{i-1\}}, I_{i_candidate})$.

If both $\text{SSIM} \geq 0.995$ and $\text{PSNR} \geq 15.0$ dB, the transformation is committed: $I_i = I_{i_candidate}$ and processing continues with $a_{\{i+1\}}$. If either condition fails, the transformation is reverted: $I_i = I_{\{i-1\}}$ and the pipeline continues to $a_{\{i+1\}}$ from the pre-agent state. In both cases, the validation outcome (accepted or reverted), the computed PSNR and SSIM values, the agent name, and the timestamp are recorded in the step-level audit log.

When the `save_intermediates` policy flag is enabled, intermediate candidate images are saved to disk at each step under `runs/{request_id}/candidates/{step_index}_{agent_name}.png`, creating a full audit trail of the transformation sequence. The final output image is saved under `runs/{request_id}/final/output.png`. A structured JSON report is written alongside it containing the full decision record, per-step metrics, per-step validation outcomes, and artifact coordinates such as the ROI bounding box.

2.11 Persistence and Real-Time Monitoring

Both processed text and processed image records are consumed by the results worker, which normalizes each record into a unified MongoDB document schema. The schema includes: `_id` (UUID string, used as the document identifier for upsert semantics), `type` ('text' or 'image'), `timestamp` (ISO 8601 UTC), `processing_duration_ms` (integer), `input_preview` (first 200 characters of the input text, or a file path for images), and `payload` (the full modality-specific processing output as a nested document).

MongoDB upsert semantics — using the UUID as the `_id` field — ensure that reprocessed records overwrite their prior versions rather than creating duplicate documents. The collection is indexed on `{type: 1, timestamp: -1}` to support the two primary query patterns: fetching the most recent N records across both modalities (sort by timestamp descending) and filtering by type for the separate text and image result views in the frontend.

The in-memory SSE buffer is implemented as a thread-safe deque with a maximum length of 200. Each time the results worker appends a record to the deque, a monotonically increasing sequence number is incremented. The FastAPI `/events` endpoint wraps the deque in an async generator that polls at 100ms intervals, emitting any records with sequence numbers greater than the last emitted sequence number as newline-delimited JSON objects prefixed with the SSE `data:` field. The frontend's EventSource listener parses each event as a JSON object and upserts it into local React state keyed by UUID, maintaining a live display of processing activity.

03. RESULTS AND DISCUSSION

3.1 Text Processing Performance

The text processing pipeline was evaluated on a representative corpus of web-sourced text comprising approximately 2,400 records across four input categories: English HTML text with spelling errors, clean English text, multilingual text (German, French, Spanish, and Sinhala), and short texts under ten words. Records were processed sequentially by a single text consumer thread on the evaluation hardware, with LLM planning and all agent executions timed independently.

3.1.1 Cleaning Success Rate and Confidence

Across all input categories, the agentic pipeline achieved a cleaning success rate of 88 to 92 percent, where a successful outcome is defined as a record for which all selected agents completed without error and the Validation Agent confirmed schema compliance. The remaining 8 to 12 percent of records were classified as partial successes due to single-agent failures — primarily translation API timeouts for low-resource languages and NER failures on texts with unusual character encodings — rather than full pipeline failures. No records experienced complete pipeline failure.

The mean overall confidence score across successfully cleaned records was 0.84 (standard deviation 0.11). The relatively low standard deviation reflects the consistency of the confidence estimation across varied input types: the confidence scores of individual agents are stable because each agent's precondition logic ensures that it only reports high confidence when its preconditions are satisfied. The LLM planner failed to produce a valid plan in approximately 2 to 5 percent of cases, owing primarily to Ollama inference timeouts under moderate server load; in all such cases the fallback plan activated without user-visible impact on the output.

3.1.2 Agent Utilization Rates

Agent utilization rates varied substantially by input category, which is precisely the adaptive behavior the framework is designed to produce. Table 2 summarizes the utilization rates across the four input categories evaluated.

Table 2 Agent Utilization Rates by Input Category

Agent	English HTML + Errors	Clean English	Multilingual	Short Text (<10w)
html_strip_agent	100%	0%	35–45%	0%
lang_detect_agent	100%	100%	100%	100%
translator_agent	0%	0%	100%	0%
spell_agent	100%	40%	0%	20%
sentence_split_agent	100%	100%	95%	0%
tokenizer_agent	100%	100%	100%	100%
ner_agent	100%	85%	90%	0%
validation_agent	100%	100%	100%	100%

The lang_detect_agent and tokenizer_agent executed on all records or near-universally across all categories, reflecting their near-universal applicability. The html_strip_agent activated on 35 to 45 percent of records in the multilingual category, consistent with the proportion of web-scraped inputs carrying markup, and on 100 percent of English HTML records by design. The spell_agent was invoked on 40 percent of clean English records but suppressed by the noise ratio threshold on the remaining 60 percent, confirming that the threshold mechanism correctly identifies near-clean inputs.

3.1.3 Latency Reduction

Table 3 presents the latency reduction achieved by selective execution compared to a fixed full-pipeline approach applying all eight agents unconditionally, measured on three representative input categories.

Table 3 Latency Reduction by Input Category

Input Category	Agents Selected	Full Pipeline Latency	Selective Latency	Reduction
English HTML + spell errors	4 of 8	1,840 ms	920 ms	~50%
Clean German text	3 of 8	1,840 ms	699 ms	~62%
Clean English (no spell)	1 of 8	1,840 ms	241 ms	~87%
Corpus mean	3.1 of 8	1,840 ms	1,105 ms	35–40%

The latency figures for the full pipeline were measured by running all eight agents unconditionally on each input, excluding LLM planning time (which is identical in both the adaptive and fixed approaches since the fixed baseline still receives the LLM call as an overhead). The mean latency reduction of 35 to 40 percent across the test corpus is a conservative estimate, as the test set was deliberately constructed to include a representative proportion of inputs that genuinely require multiple agents.

3.2 Image Processing Performance

3.2.1 Transformation Acceptance Rate

The post-transformation validation accepted 92 percent of agent outputs across all tested images, indicating that the decision engine's threshold-based selections were well-calibrated to the quality characteristics of the test set. The 8 percent of reverted transformations arose primarily from enhancement passes on images whose brightness and contrast were already close to the acceptable boundary — images with a brightness of approximately 72 or a contrast of approximately 63, just within the threshold ranges — where CLAHE occasionally pushed the SSIM below 0.995 by introducing localized over-enhancement in uniform background regions. In all such cases, the revert mechanism preserved the pre-enhancement state and the pipeline continued to the next agent if any remained.

3.2.2 Per-Agent Quality Metrics

Table 4 presents the SSIM, PSNR, and acceptance rate for each image agent, computed as means over all accepted transformations in the test set.

Table 4 Per-Agent Image Quality Metrics (Mean over Accepted Transformations)

Agent	Mean SSIM	Std SSIM	Mean PSNR (dB)	Std PSNR	Accept Rate
ROI Crop	0.997	0.001	31.4	2.1	98%
NLMeans Denoise	0.996	0.001	28.7	1.8	95%
Wiener Deblur	0.996	0.003	24.5	3.2	89%
CLAHE Enhance	0.994	0.002	25.3	2.6	91%

The ROI agent consistently produced the highest SSIM scores because cropping introduces no pixel-level distortion within the retained region — the only SSIM loss arises from the resize step that aligns the cropped output with the original dimensions for metric computation, and this loss is minimal. The Deblur agent produced the widest SSIM standard deviation, reflecting the sensitivity of Wiener deconvolution to the match between the assumed box PSF and the actual blur kernel: images blurred by a kernel close to the assumed 5x5 box receive near-perfect restoration, while images blurred by a different kernel receive partial restoration with potential ringing artifacts.

3.2.3 Quality-Aware vs Blind Processing

A direct comparison between quality-aware and blind (fixed-sequence) image processing was conducted on a set of 200 document scans with known ground truth quality labels. The blind approach applied all four agents to every image unconditionally. The quality-aware approach selectively applied agents based on the decision engine's output.

Table 5 Blind vs Quality-Aware Processing Comparison

Metric	Blind Fixed-Sequence	Quality-Aware Selective
Transformation acceptance rate	68%	92%
Mean SSIM of accepted outputs	0.995	0.996
Mean PSNR of accepted outputs (dB)	22.1	27.4
Reversion rate	32%	8%
Mean agents applied per image	4.0	2.3
Mean processing time per image (ms)	4,200	2,650

The 24 percentage point improvement in acceptance rate is the most significant result: it demonstrates that the majority of validation failures in the blind approach arose not from agent defects but from the application of agents to images that did not require them. The mean PSNR improvement of 5.3 dB for accepted outputs in the quality-aware approach reflects both the better calibration of agent selection and the revert mechanism's filtering of borderline cases.

3.3 System-Level Performance

End-to-end latency was measured from the moment of the /ingest HTTP request receipt at the API layer to the moment the processed record was confirmed written to MongoDB. This measurement includes API processing time, Kafka publish-consume latency (20–100 ms), LLM planning or metric computation time, agent execution time, validation time, result normalization, and MongoDB write time.

Table 6 System-Level Performance Summary

Metric	Design Target	Observed Range	Mean	Status
Text cleaning success	$\geq 85\%$	88–92%	90%	Met
Mean text confidence	≥ 0.80	0.73–0.94	0.84	Met
Image accept rate	$\geq 90\%$	89–95%	92%	Met
Text latency (s)	< 3.0	0.6–2.5	1.4	Met
Image latency (s)	< 10.0	2.2–5.4	3.6	Met
Text throughput (rec/s)	5–10	6–8	7.1	Met
Image throughput (rec/s)	0.5–2	0.8–1.5	1.15	Met

Metric	Design Target	Observed Range	Mean	Status
Overall error rate	< 5%	2–3%	2.4%	Well below
LLM planning latency (ms)	—	400–1200	680	Observed
Kafka routing latency (ms)	—	20–100	48	Observed

The dominant latency component for text records was the LLM planning call, which contributed 400 to 1200 ms depending on model load and input length. For image records, the dominant components were NLMeans denoising (800 to 1800 ms) and Wiener deconvolution (600 to 1400 ms). The Kafka routing latency of 20 to 100 ms per message is negligible relative to processing time but provides the decoupling and fault isolation benefits described in the architecture section.

Both pipelines exhibited linear throughput scaling with the number of Kafka consumer workers up to the point where the LLM (for text) or the CPU (for image) became the bottleneck. Adding a second text consumer worker doubled throughput from approximately 3.5 records per second to 7.1 records per second, confirming the linear scaling property. Adding a third worker produced only a 20 percent additional improvement because LLM inference capacity became the binding constraint.

3.4 Discussion

3.4.1 Interpretation of Core Findings

The 35 to 40 percent mean latency reduction on the text side is the primary quantitative result for RQ1. This reduction did not come from faster agents — every agent executes at the same speed as it would in a fixed pipeline. The gain came entirely from not running agents that had nothing to do. This is the fundamental argument for adaptive preprocessing: the cost of running a spell corrector on text with no misspellings is not zero — it is the full execution time of the correction loop plus the risk of an inadvertent substitution. The LLM planner eliminates that cost by inspecting the input first and identifying that the spell agent should not be invoked.

The practical implication is significant: a preprocessing pipeline handling millions of records per day — a realistic production scale — achieves a 35 to 40 percent reduction in total compute time without any change to the agents themselves and without any reduction in preprocessing quality for records that genuinely require correction. At production scale, this translates directly to reduced infrastructure cost and reduced processing latency, both of which have downstream effects on model training frequency and product responsiveness.

The 24-percentage point improvement in image validation acceptance rate over the blind approach is the primary quantitative result for RQ3. It demonstrates that the quality-aware selection mechanism prevents the application of agents to images that do not require them, and that the revert mechanism catches the residual cases where the metric-based gating was insufficient. The two mechanisms are complementary: the decision engine eliminates the majority of unnecessary agent applications before execution, while the validator catches the minority that pass the metric thresholds but nonetheless produce suboptimal transformations.

3.4.2 Asymmetry Between Text and Image Decision Mechanisms

The deliberate asymmetry between LLM-based text planning and rule-based image planning is the most conceptually significant design choice in the framework, and the results support it directly. Text preprocessing decisions resist quantification because they require understanding the content of the text. Whether a document needs HTML stripping can be determined by scanning for angle brackets, but whether it needs spell correction depends on the language register — 'gonna' is not misspelled in informal English — and whether it needs NER depends on whether it contains named entities at all, which requires reading it. A rule-based engine attempting to encode these decisions would need dozens of heuristics and would still fail on edge cases. An LLM handles them naturally because it can read and interpret text.

Image quality decisions, by contrast, are inherently quantitative. The question 'is this image blurry?' is answered definitively and reproducibly by computing the Laplacian variance. There is no ambiguity about whether a Laplacian variance of 85 is below the threshold of 120. The decision is explicit, its rationale is grounded in a validated signal processing operator, and its output is deterministic across repeated calls on the same input. Substituting an LLM for this decision would add 400 to 1200 ms of inference latency, introduce non-determinism, and provide no benefit — the LLM cannot access pixel-level information and would need to rely on a verbal description of image quality that would be less accurate than the direct metric computation.

This asymmetry implies a general design principle for agentic preprocessing systems: the appropriate decision mechanism for each modality is the one that matches the nature of the information being processed. Where the decision requires reading and interpreting content, an LLM is appropriate. Where the decision can be derived from measurable scalar properties of the signal, a rule-based engine is more accurate, faster, and fully reproducible. The framework demonstrates both patterns operating together within a single production-grade infrastructure.

3.4.3 Architectural Implications of Kafka-Based Decoupling

The Kafka backbone deserves analysis beyond its contribution to throughput. In a synchronous preprocessing pipeline, the API endpoint blocks while each record is processed — a file upload containing 10,000 rows would hold an HTTP connection open for the entire duration of row-by-row processing, potentially tens of minutes. By splitting files at ingestion time and publishing each row as an independent Kafka message, the framework converts a single long-running synchronous operation into a burst of small asynchronous tasks. The API endpoint returns immediately with a request UUID, and the client receives individual row results progressively via the SSE stream as each row is processed. This is a qualitatively different user experience and a meaningfully different resource utilization pattern.

The consumer group model provides fault isolation at the component level. A crash in the image consumer does not affect the text consumer because they subscribe to separate topics with independent consumer group offsets. When the image consumer restarts, it resumes from the last committed offset, reprocessing any messages that were in-flight at the time of the crash. This at-least-once delivery guarantee, combined with MongoDB's upsert semantics using the message UUID as the document key, ensures that each record is processed exactly once in the final database even if it is processed multiple times by the consumer.

3.4.4 Broader Applicability

The framework described here is not specific to the particular agents it currently contains. The agent interface — a stateless transformation function with a confidence score, a `should_run` precondition flag, and a changed output flag — is generic enough that any preprocessing step can be registered with the text or image pipeline. On the text side, this could include domain-specific normalization steps such as currency format unification, date parsing, technical acronym expansion, or PII detection and redaction. On the image side, it could include orientation correction, watermark detection, text region localization for OCR preprocessing, or color calibration for scientific imaging.

The same architecture could be adapted to audio preprocessing, where quality metrics such as signal-to-noise ratio and clipping rate could drive agent selection for noise reduction, normalization, and resampling. It could equally apply to tabular data preprocessing, where column-level statistical profiles — null rate, cardinality, distribution skewness — could inform decisions about imputation strategy, outlier handling, and feature encoding. In each case, the key question is the same: can the need for a preprocessing operation be diagnosed from measurable properties of the input? Where the answer is yes, a rule-based decision engine is appropriate. Where the answer requires reading and interpreting the content, an LLM planner is appropriate.

3.4.5 Limitations and Honest Acknowledgements

Three limitations of the current implementation merit direct discussion. First, the Wiener deconvolution assumes a uniform box PSF. This simplification holds reasonably for defocus blur but breaks down entirely for motion blur, which produces a directional PSF whose orientation and extent depend on the direction and speed of movement at exposure time. An image blurred by diagonal camera shake will be inadequately restored by a symmetric 5x5 box filter, and the restoration artifacts — ringing in the frequency domain — may be more distracting than the original blur. Modern neural deblurring approaches address this by learning to estimate the PSF implicitly from the blurred image itself, but integrating a learned deblurring model adds GPU requirements and significantly increases deployment complexity for the lightweight deployment target of this framework.

Second, the LLM planner introduces variable and difficult-to-bound latency. Under light load, the Ollama inference call completes in 400 to 600 ms. Under contention from concurrent requests, this rises to over 1200 ms — a non-trivial fraction of the total text processing budget. There is currently no mechanism to detect slow-but-eventually-successful LLM responses and substitute the fallback plan proactively. A timeout-based approach with graceful degradation would partially address this, allowing the system to trade plan quality for latency guarantees under heavy load. The tradeoff — sacrificing adaptive planning to meet latency targets — is qualitatively different from a hard failure, but it reduces the adaptive advantage of the framework to the level of the conservative fallback plan.

Third, and most fundamentally, neither the LLM planner nor the rule-based image decision engine learns from experience. The image quality thresholds were established by examining a representative sample of inputs and are not updated as the system processes more data. The LLM planner selects agents based on its pre-training distribution and produces the same plan for structurally similar inputs regardless of whether previous plans for that input type led to good outcomes. Incorporating a lightweight feedback loop — tracking which agent plans produce the highest downstream model performance on a held-out validation set and using this signal to update either the LLM prompt or the rule thresholds — would allow the system to improve continuously over its deployment lifetime. This is the most significant architectural gap relative to a fully agentic system in the ReAct sense, where the agent can observe the consequences of its actions and adjust future behavior accordingly.

04. CONCLUSION

4.1 Conclusions

This dissertation has presented an agentic multi-modal preprocessing framework that addresses the fundamental inefficiency of static, fixed-sequence preprocessing pipelines through a combination of LLM-guided dynamic planning, quantitative rule-based quality assessment, per-step validation with automatic revert capability, and a Kafka-based streaming infrastructure that unifies both modalities under a single API.

On the text side, the OllamaPlanner-based orchestration demonstrated a mean latency reduction of 35 to 40 percent compared to a fixed full-pipeline approach, with per-category reductions of up to 87 percent for inputs that require only a single agent. The overall text cleaning success rate of 88 to 92 percent and mean confidence of 0.84 confirm that the adaptive approach does not sacrifice preprocessing quality for efficiency: records that require multiple agents receive them, while records that require fewer receive proportionally fewer.

On the image side, the rule-based decision engine driven by Laplacian variance, mean luminance, and contrast standard deviation achieved a 92 percent post-transformation validation acceptance rate, compared to 68 percent for a blind fixed-sequence baseline — a 24 percentage point improvement that directly validates the core thesis of this work: selective, quality-driven agent application produces better outcomes than uniform application across all inputs.

The post-transformation PSNR/SSIM validation layer provided a critical second safety net, catching the 8 percent of cases where the metric-based gating was insufficient to prevent a suboptimal transformation. Without this layer, 8 percent of image outputs in the quality-aware pipeline would have experienced SSIM degradation below acceptable levels — a rate that drops to zero when the revert mechanism is active. The combination of proactive selection and reactive validation achieves a level of output quality assurance that neither mechanism alone can provide.

The Apache Kafka streaming backbone successfully decoupled ingestion, routing, processing, and persistence across both modalities, enabling asynchronous parallel processing with sub-3-second text latency and sub-6-second image latency even for the most compute-intensive inputs. The consumer group scaling property was validated empirically, with throughput doubling when a second text consumer was added. The SSE-based real-time monitoring frontend delivered a qualitatively different user experience from polling-based alternatives, providing progressive result delivery for large batch inputs without holding HTTP connections open.

All stated design targets were met or exceeded across every measured metric. The framework demonstrates that combining LLM-guided planning for semantically complex decisions with quantitative rule-based planning for measurable quality decisions, unified under a streaming infrastructure with per-step validation, produces a demonstrably more adaptive, resource-efficient, and auditable alternative to static preprocessing pipelines.

The deliberate asymmetry between text and image decision mechanisms — LLM for text, rule-based for images — is the framework's most conceptually significant design contribution. It demonstrates a general design principle: the right orchestration mechanism for a given modality is determined by whether the decision requires content interpretation (favouring LLM) or whether it can be derived from scalar signal properties (favouring rule-based systems). This principle is applicable beyond the specific modalities and agents described in this dissertation.

4.2 Recommendations and Future Work

The evaluation and implementation experience with this framework suggest several concrete directions for future improvement and extension:

1. **Neural PSF Estimation for Motion Deblur:** The current Wiener deconvolution component assumes a uniform box PSF, which is appropriate for defocus blur but inadequate for motion blur. Future work should integrate a lightweight learned deblur network — such as a blind deconvolution model trained on motion blur datasets — that can estimate the PSF from the blurred observation itself. This would significantly improve the Deblur Agent's acceptance rate for motion-blurred inputs while retaining the Wiener-based approach as a computationally efficient fallback.
2. **LLM Latency Bounding via Proactive Fallback:** The current implementation waits for the LLM planning call to complete or time out before activating the fallback plan. A more robust approach would implement a speculative execution model in which the fallback plan begins executing in a parallel thread while the LLM is still planning. If the LLM response arrives before the fallback completes, the fallback is cancelled and the LLM plan is used. If the fallback completes first, it is committed and the LLM response is discarded. This approach would reduce P99 latency significantly without sacrificing plan quality for the majority of requests.
3. **Adaptive Threshold Learning via Feedback Loop:** The most significant architectural gap relative to a fully agentic system is the absence of a learning component. Future work should implement a lightweight feedback loop that tracks the downstream impact of preprocessing decisions — for example, OCR character error rate for document image pipelines, or NER F1 score for text pipelines — and uses this signal to update the image quality thresholds and the LLM planner prompt. Even a simple hill-climbing algorithm operating

on a small held-out validation set would allow the system to discover domain-specific thresholds that outperform the manually-set defaults.

4. Extended Agent Catalogues: The generic agent interface makes it straightforward to add new agents without modifying the orchestration logic. On the text side, domain-specific additions could include currency format normalization, date parsing and standardization, technical acronym expansion, PII detection and redaction (complementing the NER agent), and readability scoring. On the image side, additions could include orientation correction via EXIF metadata or content-based detection, watermark detection and removal, text region localization for OCR preprocessing, and color calibration for scientific or medical imaging applications.

5. Multi-Modal Extension to Audio and Tabular Data: The framework's architecture generalizes naturally to additional modalities. For audio preprocessing, signal-to-noise ratio and clipping rate could drive agent selection for noise reduction, normalization, silence trimming, and resampling. For tabular data, column-level statistical profiles — null rate, cardinality, distribution skewness, inter-column correlation — could drive decisions about imputation strategy, outlier handling, categorical encoding, and feature scaling. The Kafka routing layer, the validation protocol, and the MongoDB persistence infrastructure would remain unchanged in both cases.

6. Distributed Deployment and GPU Acceleration: The current evaluation was conducted on a single-node development machine. Future work should validate the linear throughput scaling property at production ingestion volumes using a multi-broker Kafka cluster and multiple consumer worker processes. NLMeans denoising and Wiener deconvolution are both parallelizable at the image level and could benefit significantly from GPU acceleration using CUDA-backed implementations, potentially increasing image throughput by an order of magnitude.

7. Explainability Interface for Audit and Governance: The framework already generates per-step audit trails including agent selection rationale, metric values, confidence scores, and validation outcomes. A natural extension would expose this audit information through a dedicated governance interface in the React frontend, allowing compliance officers and data engineers to query, filter, and export preprocessing decisions for any record. This capability is particularly valuable in regulated industries where data transformation provenance is a compliance requirement.

REFERENCES

- [1] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. NetDB Workshop, 2011.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Representations (ICLR), 2023.
- [3] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," arXiv preprint arXiv:2303.17580, 2023.
- [4] A. V., G. R. Gangadharan, and R. Buyya, "Agentic artificial intelligence (AI): Architectures, taxonomies, and evaluation of large language model agents," arXiv preprint arXiv:2601.12560, 2026.
- [5] M. Lui and T. Baldwin, "langid.py: An off-the-shelf language identification tool," in Proc. ACL 2012 System Demonstrations, pp. 25–30, 2012.
- [6] T. Kiss and J. Strunk, "Unsupervised multilingual sentence boundary detection," Computational Linguistics, vol. 32, no. 4, pp. 485–525, 2006, doi: 10.1162/coli.2006.32.4.485.
- [7] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit. Sebastopol, CA: O'Reilly Media, 2009.
- [8] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," Zenodo, 2020, doi: 10.5281/zenodo.1212303.
- [9] ITU-R, Recommendation ITU-R BT.601-7: Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios, 2011.
- [10] S. Pertuz, D. Puig, and M. A. Garcia, "Analysis of focus measure operators for shape-from-focus," Pattern Recognition, vol. 46, no. 5, pp. 1415–1432, 2013, doi: 10.1016/j.patcog.2012.11.011.
- [11] K. J. Zuiderveld, "Contrast limited adaptive histogram equalization," in Graphics Gems IV, P. S. Heckbert, Ed. San Diego, CA: Academic Press, 1994, pp. 474–485.
- [12] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in Proc. 6th Int. Conf. Computer Vision (ICCV), pp. 839–846, 1998, doi: 10.1109/ICCV.1998.710815.
- [13] A. Buades, B. Coll, and J.-M. Morel, "A non-local algorithm for image denoising," in Proc. IEEE CVPR, vol. 2, pp. 60–65, 2005, doi: 10.1109/CVPR.2005.38.
- [14] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679–698, 1986, doi: 10.1109/TPAMI.1986.4767851.

- [15] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: 10.1109/TIP.2003.819861.
- [16] S. Ramírez, *FastAPI Documentation: Modern, Fast (High-Performance) Web Framework for Building APIs with Python*, 2018.
- [17] Pydantic, *Pydantic Documentation: Data Validation Using Python Type Hints*. [Online]. Available: <https://docs.pydantic.dev>
- [18] S. Bradshaw, E. Brazil, and K. Chodorow, *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2019.
- [19] Google Cloud, *Cloud Translation API Documentation*. [Online]. Available: <https://cloud.google.com/translate/docs>
- [20] F. Orieux, J.-F. Giovannelli, and T. Rodet, "Bayesian estimation of regularization and point spread function parameters for Wiener-Hunt deconvolution," *Journal of the Optical Society of America A*, vol. 27, no. 7, pp. 1593–1607, 2010.